

```
const byte SENSOR_PIN = A0;    // Analog read pin for high-pass filter signal
const byte RELAY_PIN = 13;     // Control Pin: LOW = Closed (Normal), HIGH = Open
                                // (Cutoff)
const byte GENERATOR_PIN = 11; // Carrier wave output (Timer 2)
const byte STAGE_PIN = 9;      // Input pin to advance calibration stages
                                // (INPUT_PULLUP)
const byte ALARM_PIN = 8;      // Proximity early-warning alarm output pin
```

```
// ——— ASYMMETRICAL FILTER CONFIGURATION ———
```

```
float smoothedAmp = 0;
const float SMOOTH_DOWN = 0.88; // Faster tracking when value drops (hand
approaching)
const float SMOOTH_UP = 0.15;   // Slower tracking when value rises (hand retreating)
```

```
// ——— CALIBRATION VARIABLES & SENSITIVITY TWEAKS ———
```

```
unsigned long baselineClosed = 0;
unsigned long thresholdOpenIt = 0;
unsigned long thresholdAlarm = 0;
unsigned long baselineOpen = 0;
unsigned long thresholdCloseIt = 0;
```

```
// Sensitivity percentages
```

```
const float CUTOFF_SENSITIVITY = 0.055;
const float ALARM_SENSITIVITY = 0.025;
const float RECONNECT_SENSITIVITY = 0.025;
```

```
// Dynamic Adaptive Drift
```

```
const float DRIFT_SLOW_PCT = 0.003;    // 0.3% small drift threshold
const float DRIFT_FAST_PCT = 0.030;    // 3.0% large drift threshold
```

```
// Drift Variables when Circuit Closed
```

```
const unsigned long CLOSED_DRIFT_SLOW_TIME = 500; // 500ms for small drift  
when closed
```

```
const unsigned long CLOSED_DRIFT_FAST_TIME = 2000; // 2000ms (2s) for big drift  
(< cutoff) when closed
```

```
// Drift Variables when Circuit Opened
```

```
const unsigned long OPEN_DRIFT_SLOW_TIME = 1000; // 1000ms (1s) for small drift  
when open
```

```
const unsigned long OPEN_DRIFT_FAST_TIME = 30000; // 30000ms (30s) for big drift  
when open
```

```
// Dynamic drift tracking timers
```

```
unsigned long slowDriftStartTime = 0;
```

```
unsigned long fastDriftStartTime = 0;
```

```
float currentDriftRatio = 1.0;
```

```
bool circuitIsClosed = true;
```

```
unsigned long lastSerialTime = 0;
```

```
// RECONNECT TIMER VARIABLES
```

```
unsigned long clearStartTime = 0;
```

```
const unsigned long RECONNECT_DELAY = 3000; // 3000ms = 3 seconds
```

```
int absMax = 0;
```

```
int absMin = 1023;
```

```
// Helper function to recalculate closed state thresholds
```

```
void updateClosedThresholds(unsigned long newBase) {
```

```
    baselineClosed = (SMOOTH_DOWN * newBase) + ((1.0 - SMOOTH_DOWN) *  
newBase);
```

```
    thresholdAlarm = baselineClosed - (baselineClosed * ALARM_SENSITIVITY);
```

```
    thresholdOpenIt = baselineClosed - (baselineClosed * CUTOFF_SENSITIVITY);
```

```
}
```

```
// Helper function to recalculate open state thresholds
```

```
void updateOpenThresholds(unsigned long newBase) {
```

```
    baselineOpen = newBase;
```

```
    thresholdCloseIt = baselineOpen - (baselineOpen * RECONNECT_SENSITIVITY);
```

```
}
```

```
// Wait a bit for calibration
```

```
void waitForStageAdvance() {
```

```
    while (digitalRead(STAGE_PIN) == HIGH) {
```

```
        delay(1);
```

```
    }
```

```
    delay(50); // Debounce
```

```
    while (digitalRead(STAGE_PIN) == LOW) {
```

```
        delay(1);
```

```
    }
```

```
    delay(50); // Debounce
```

```
}
```

```
void setup() {
```

```
    pinMode(RELAY_PIN, OUTPUT);
```

```
    digitalWrite(RELAY_PIN, LOW); // Inverted logic: LOW = CLOSED (Normal operation)
```

```
    circuitIsClosed = true;
```

```
pinMode(ALARM_PIN, OUTPUT);  
digitalWrite(ALARM_PIN, LOW); // Initialize alarm to OFF
```

```
pinMode(GENERATOR_PIN, OUTPUT);  
pinMode(STAGE_PIN, INPUT_PULLUP);
```

```
// TIMER 2 SQUARE WAVE GENERATION  
TCCR2A = _BV(COM2A0) | _BV(WGM21);  
TCCR2B = _BV(CS20); // No prescaling (N=1)  
OCR2A = 7;          // 1MHz square signal
```

```
Serial.begin(115200);
```

```
// Overclock ADC prescaler to 32 for fast loop cycles  
ADCSRA = (ADCSRA & 0xF8) | 0x05;
```

```
delay(800);  
Serial.println(F("\n====="));  
Serial.println(F("  ASYMMETRICAL SENSOR SYSTEM CALIBRATION  "));  
Serial.println(F("====="));
```

```
// STAGE 1: LIVE CLOSED-STATE CALIBRATION  
Serial.println(F("\n[STAGE 1] Calibrating CLOSED State (Relay is ON/LOW)..."));  
Serial.println(F("-> Leave hands clear or mimic normal running noise."));  
Serial.println(F("-> Ground Pin 9 to lock baselines and proceed."));
```

```
int calMinClosed = 1023;  
int calMaxClosed = 0;
```

```
unsigned long localSerialTime = 0;
unsigned long autoCalibration = millis();
unsigned long autoCalibrationTime = 2000;
```

```
while (digitalRead(STAGE_PIN) == HIGH && millis() - autoCalibration <
autoCalibrationTime) {
```

```
    int sample = analogRead(SENSOR_PIN);
    if (sample < calMinClosed) calMinClosed = sample;
    if (sample > calMaxClosed) calMaxClosed = sample;
```

```
    if (millis() - localSerialTime >= 150) {
        localSerialTime = millis();
        Serial.print(F("Live ADC: ")); Serial.print(sample);
        Serial.print(F(" | Min: ")); Serial.print(calMinClosed);
        Serial.print(F(" | Max: ")); Serial.println(calMaxClosed);
    }
}
```

```
unsigned long openxvalue = (calMaxClosed + calMinClosed) / 2;
updateClosedThresholds(openxvalue);
```

```
Serial.println(F(">> STAGE 1 LOCKED. Release Pin 9 wire..."));
// waitForStageAdvance();
```

```
// STAGE 2: LIVE OPEN-STATE CALIBRATION
```

```
digitalWrite(RELAY_PIN, HIGH); // Open the relay circuit
delay(1000);                    // Let power rail voltage stabilize
```

```
Serial.println(F("\n[STAGE 2] Calibrating OPEN State (Relay is OFF/HIGH)..."));
```

```
Serial.println(F("-> Ground Pin 9 when finished tracking open-state noise."));
```

```
int calMinOpen = 1023;
```

```
int calMaxOpen = 0;
```

```
localSerialTime = 0;
```

```
autoCalibration = millis();
```

```
while (digitalRead(STAGE_PIN) == HIGH && millis() - autoCalibration < autoCalibrationTime) {
```

```
    int sample = analogRead(SENSOR_PIN);
```

```
    if (sample < calMinOpen) calMinOpen = sample;
```

```
    if (sample > calMaxOpen) calMaxOpen = sample;
```

```
    if (millis() - localSerialTime >= 150) {
```

```
        localSerialTime = millis();
```

```
        Serial.print(F("Live Sag ADC: ")); Serial.print(sample);
```

```
        Serial.print(F(" | Min: ")); Serial.print(calMinOpen);
```

```
        Serial.print(F(" | Max: ")); Serial.println(calMaxOpen);
```

```
    }
```

```
}
```

```
unsigned long closexvalue = (calMinOpen + calMaxOpen) / 2;
```

```
updateOpenThresholds(closexvalue);
```

```
Serial.println(F(">> STAGE 2 LOCKED. Release Pin 9 wire..."));
```

```
// waitForStageAdvance();
```

```
// RESUME NORMAL RUNTIME OPERATION
```

```
digitalWrite(RELAY_PIN, LOW); // Re-close the relay to start safe
```

```
circuitIsClosed = true;
```

```
smoothedAmp = baselineClosed;
```

```
delay(1000);
```

```
Serial.println(F("\n--- CALIBRATION PROFILES SET ---"));
```

```
Serial.print(F("Closed Base: ")); Serial.print(baselineClosed);
```

```
Serial.print(F(" | Alarm At: ")); Serial.print(thresholdAlarm);
```

```
Serial.print(F(" | Cutoff At: ")); Serial.println(thresholdOpenIt);
```

```
Serial.print(F("Open Base: ")); Serial.print(baselineOpen);
```

```
Serial.print(F(" | Reconnect At: ")); Serial.println(thresholdCloseIt);
```

```
Serial.println(F("=====\n"));
```

```
delay(1500);
```

```
absMax = 0;
```

```
absMin = 1023;
```

```
}
```

```
// Function to calculate Peak-to-Peak amplitude while discarding rogue spikes
```

```
int readCleanRawAmp(byte pin) {
```

```
    const int SAMPLE_COUNT = 64;
```

```
    int samples[SAMPLE_COUNT];
```

```
    long sum = 0;
```

```
    for (int i = 0; i < SAMPLE_COUNT; i++) {
```

```
        samples[i] = analogRead(pin);
```

```
        sum += samples[i];
```

```
    }
```

```
    float roughMean = (float)sum / SAMPLE_COUNT;
```

```
float weightedSum = 0;
```

```
float totalWeight = 0;
```

```
const float k = 2.5;
```

```
for (int i = 0; i < SAMPLE_COUNT; i++) {
```

```
    float distance = (samples[i] - roughMean) / k;
```

```
    float weight = 1.0 / (1.0 + (distance * distance));
```

```
    weightedSum += samples[i] * weight;
```

```
    totalWeight += weight;
```

```
}
```

```
return (int)(weightedSum / totalWeight);
```

```
}
```

```
void loop() {
```

```
    int rawAmp = readCleanRawAmp(SENSOR_PIN);
```

```
    // 1. Asymmetrical Filtering Processing
```

```
    if (rawAmp < smoothedAmp) {
```

```
        smoothedAmp = (SMOOTH_DOWN * rawAmp) + ((1.0 - SMOOTH_DOWN) *  
smoothedAmp);
```

```
    } else {
```

```
        smoothedAmp = (SMOOTH_UP * rawAmp) + ((1.0 - SMOOTH_UP) *  
smoothedAmp);
```

```
    }
```

```
    // 2. Active State Baseline Drift Analysis
```

```
    unsigned long activeBase = circuitIsClosed ? baselineClosed : baselineOpen;
```

```

if (activeBase > 0) {
    currentDriftRatio = smoothedAmp / (float)activeBase;
} else {
    currentDriftRatio = 1.0;
}

float absDriftDev = fabs(currentDriftRatio - 1.0); // Absolute drift percentage from 1.000

// Select state-specific target delay values
unsigned long targetSlowTime = circuitIsClosed ? CLOSED_DRIFT_SLOW_TIME :
OPEN_DRIFT_SLOW_TIME;
unsigned long targetFastTime = circuitIsClosed ? CLOSED_DRIFT_FAST_TIME :
OPEN_DRIFT_FAST_TIME;

// Check if within safe limits to drift (does NOT trigger cutoff in CLOSED, or reconnect
in OPEN)
bool withinSafeLimits = circuitIsClosed ? (smoothedAmp >= thresholdOpenIt) :
(smoothedAmp <= thresholdCloseIt);

if (withinSafeLimits) {
    // SMALL/SLOW DRIFT (0.1% to 3.0%)
    if (absDriftDev >= DRIFT_SLOW_PCT && absDriftDev < DRIFT_FAST_PCT) {
        fastDriftStartTime = 0; // Cancel fast/large drift counter

        if (slowDriftStartTime == 0) {
            slowDriftStartTime = millis();
        } else if (millis() - slowDriftStartTime >= targetSlowTime) {
            // Adapt baseline and recalculate thresholds for active state
            unsigned long newBase = (unsigned long)(smoothedAmp + 0.5);
            if (circuitIsClosed) {

```

```

        updateClosedThresholds(newBase);
    } else {
        updateOpenThresholds(newBase);
    }
    slowDriftStartTime = 0;
}
}
// LARGE/FAST DRIFT (>= 3.0%)
else if (absDriftDev >= DRIFT_FAST_PCT) {
    slowDriftStartTime = 0; // Cancel small drift counter

    if (fastDriftStartTime == 0) {
        fastDriftStartTime = millis();
    } else if (millis() - fastDriftStartTime >= targetFastTime) {
        // Signal sustained at shifted level -> accept as new base
        unsigned long newBase = (unsigned long)(smoothedAmp + 0.5);
        if (circuitIsClosed) {
            updateClosedThresholds(newBase);
        } else {
            updateOpenThresholds(newBase);
        }
        fastDriftStartTime = 0;
    }
}
else {
    // Signal is stable around baseline (< 0.1% drift) -> reset timers
    slowDriftStartTime = 0;
    fastDriftStartTime = 0;
}

```

```
} else {  
    // Cutoff or reconnect limit reached -> pause drift tracking  
    slowDriftStartTime = 0;  
    fastDriftStartTime = 0;  
}
```

```
// 3. Dual-State System Control Machine
```

```
if (circuitIsClosed) {  
    // CIRCUIT CLOSED MODE (Normal Working State)  
  
    // Handle Early Warning Proximity Alarm  
    if (smoothedAmp < thresholdAlarm) {  
        digitalWrite(ALARM_PIN, HIGH);  
    } else {  
        digitalWrite(ALARM_PIN, LOW);  
    }  
  
    // Handle Main Safety Trip  
    if (smoothedAmp < thresholdOpenIt) {  
        circuitIsClosed = false;  
        digitalWrite(RELAY_PIN, HIGH); // HIGH opens relay  
        digitalWrite(ALARM_PIN, LOW);  
        smoothedAmp = baselineOpen;  
        clearStartTime = 0;  
        slowDriftStartTime = 0;  
        fastDriftStartTime = 0;  
        absMax = 0;  
        absMin = 1023;  
    }  
}
```

```

}
else {
    // CIRCUIT OPEN MODE (Tripped/Safe State)
    digitalWrite(ALARM_PIN, LOW);

    if (smoothedAmp > thresholdCloseIt) {
        if (clearStartTime == 0) {
            clearStartTime = millis();
        }

        if (millis() - clearStartTime >= RECONNECT_DELAY) {
            circuitIsClosed = true;
            digitalWrite(RELAY_PIN, LOW); // LOW closes relay
            smoothedAmp = baselineClosed;
            clearStartTime = 0;
            slowDriftStartTime = 0;
            fastDriftStartTime = 0;
            absMax = 0;
            absMin = 1023;
        }
    }
    else {
        clearStartTime = 0;
    }
}

```

// 4. Formatted Telemetry Reporting (Every 120ms)

```

if (millis() - lastSerialTime >= 120) {
    lastSerialTime = millis();
}

```

```
unsigned long activeTarget = circuitIsClosed ? thresholdOpenIt : thresholdCloseIt;
```

```
Serial.print(F("R_Min: "));    Serial.print(absMin);  
Serial.print(F(" | R_Max: "));  Serial.print(absMax);  
Serial.print(F(" | Smooth: ")); Serial.print(smoothedAmp, 1);  
Serial.print(F(" | DriftRatio: ")); Serial.print(currentDriftRatio, 3);  
Serial.print(F(" | TargetThresh: ")); Serial.print(activeTarget);  
Serial.print(F(" | ALARM: "));  Serial.print(thresholdAlarm);
```

```
// Reconnect Timer Telemetry
```

```
Serial.print(F(" | Timer: "));
```

```
if (!circuitIsClosed && clearStartTime > 0) {
```

```
    float secondsLeft = (RECONNECT_DELAY - (millis() - clearStartTime)) / 1000.0;
```

```
    Serial.print(secondsLeft, 1); Serial.print(F("s"));
```

```
} else if (!circuitIsClosed) {
```

```
    Serial.print(F("Hand Near"));
```

```
} else {
```

```
    Serial.print(F("OFF"));
```

```
}
```

```
Serial.print(F(" | State: "));    Serial.println(circuitIsClosed ? F("CLOSED") :
```

```
F("OPEN"));
```

```
}
```

```
}
```